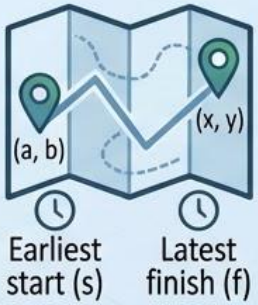
Self-Driving Rides Optimization

(Google Hash Code 2018)

Gonçalo Santos
João Ferreira
Manuel Pedro

Problem Specification

Problem Description

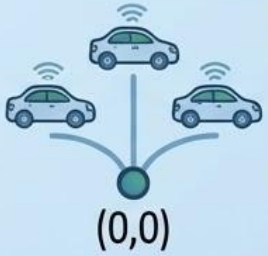


Assign a set of rides to a fleet of self-driving cars.

Each ride has:

- Start position (a, b)
- End position (x, y)
- Earliest start time (s)
- Latest finish time (f)

Vehicles



- All cars start at position (0, 0).
- Cars execute rides sequentially.
- Each car can perform multiple rides.

Objective

Maximize the total score.

Scoring

Score = ride distance + bonus if the ride starts exactly at its earliest start time.

Constructive Heuristics (Greedy)

Heuristic Approaches

Greedy	→	immediate profit
Smart Greedy	→	cost + waiting + bonus
Randomized Greedy	→	diversity (GRASP)

Local Search Metaheuristics

Hill Climbing

only improves

Simulated Annealing

accepts worse solutions

Operators:

Swap

Relocate

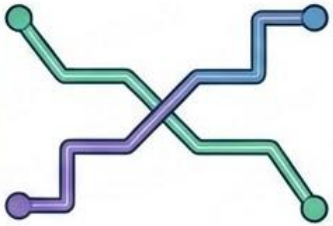
Genetic Algorithm

Population of solutions

Initial set of candidate routes

→

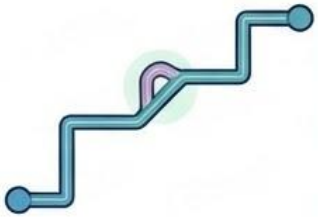
Crossover → combine routes



Exchange genetic material between parents

→

Mutation → small changes



Introduce random variations

→

Elitism → keep best solutions

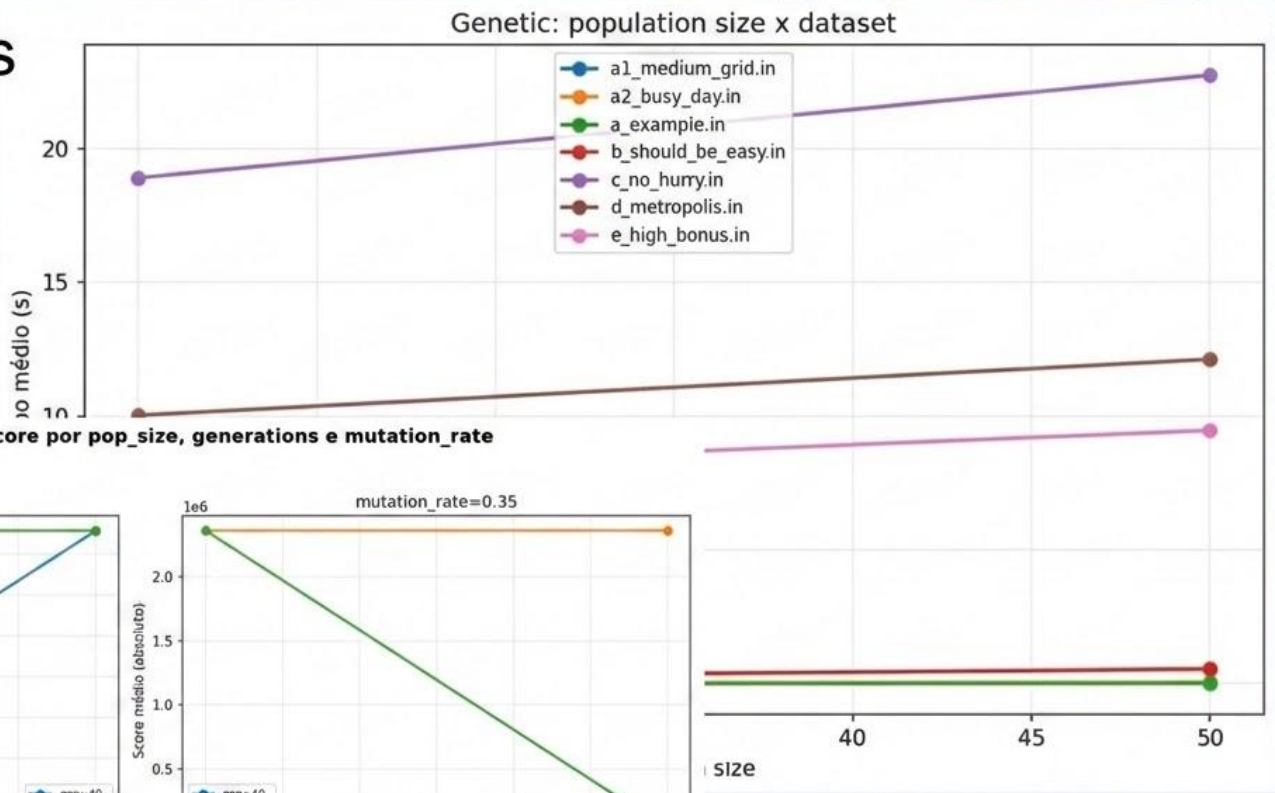
Carry over top performers to next generation

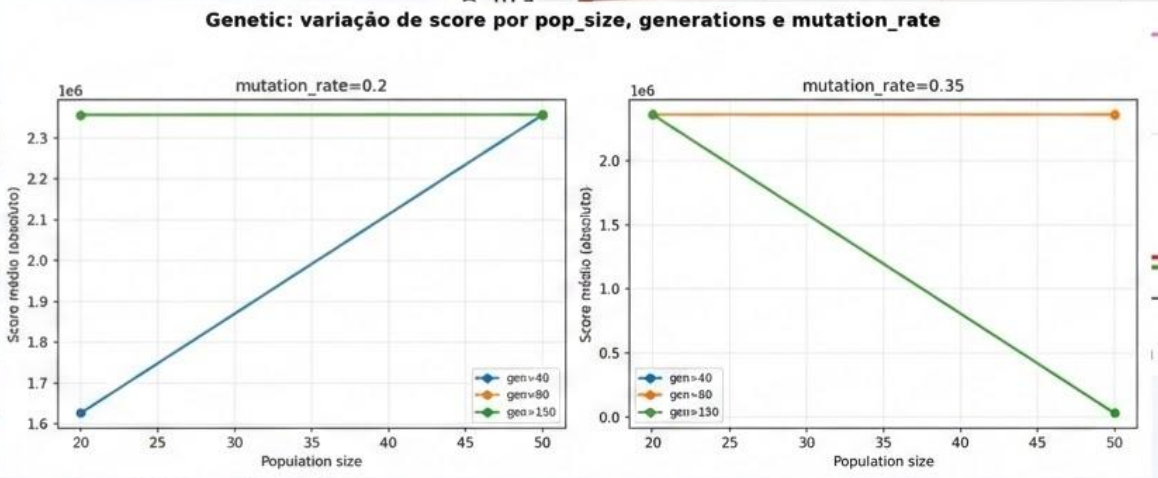
Parameter Analysis

Population: 20 vs 100 vs 300

Best trade-off: 100 individuals

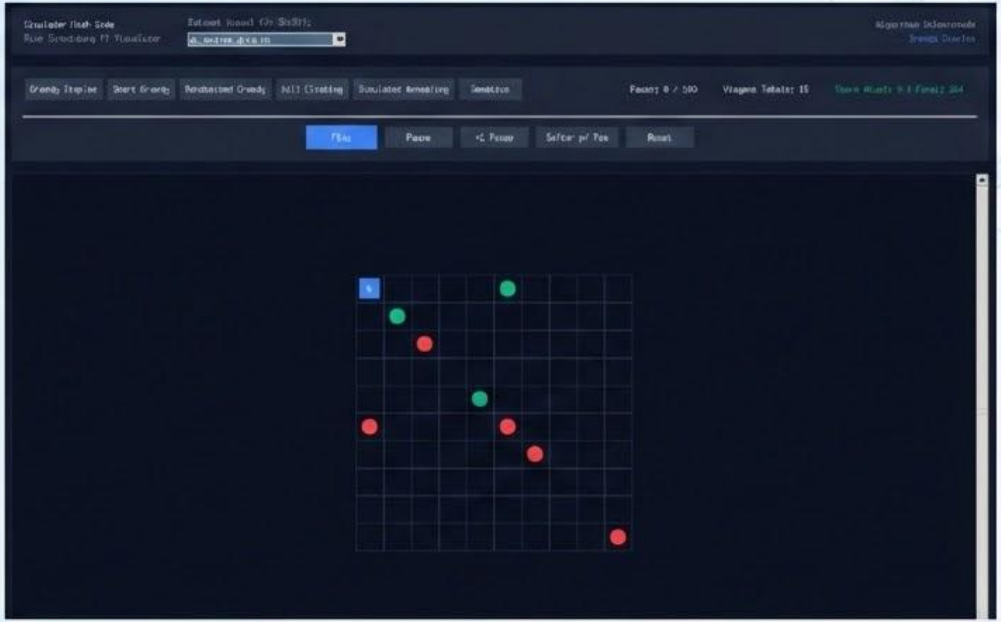
More time ≠ much better results





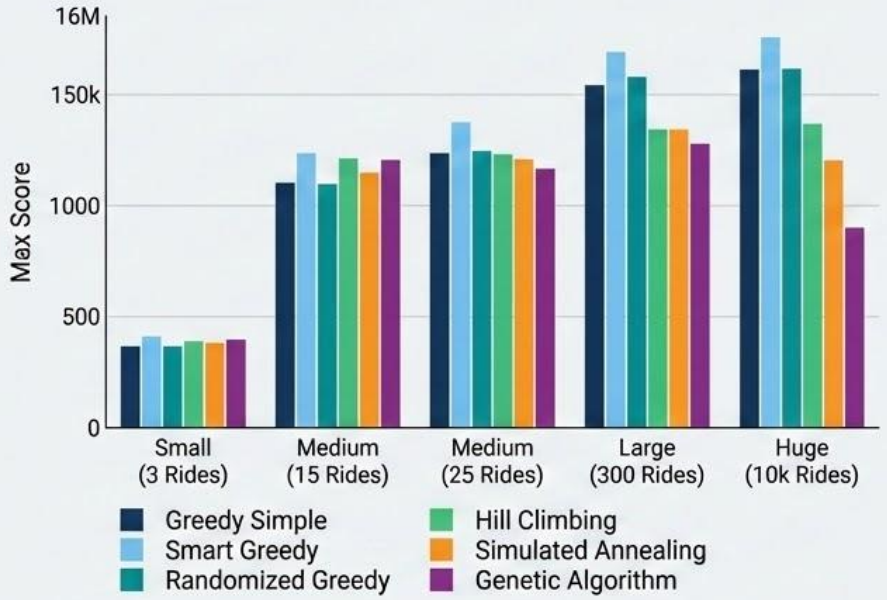
Interface & Visualization

- Dark mode UI
- Visualization up to 30×30 grid
- Vehicle states:
  - Idle
  - Waiting
  - Busy

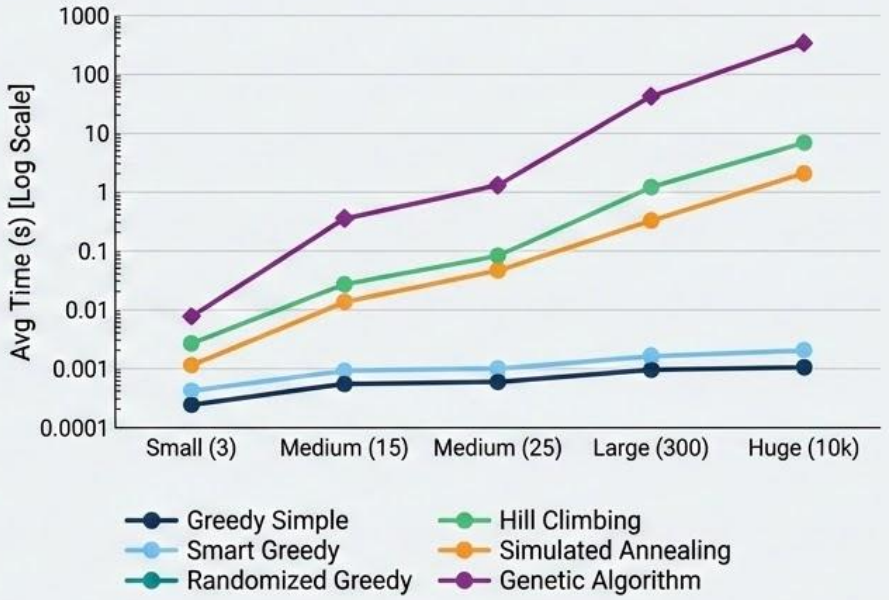


Experimental Results

Max Score Comparison (Higher is Better)



Avg Execution Time (Lower is Better) - Log Scale



- **Key Observations:** Smart Greedy consistently achieves near-optimal scores for large datasets.
- **Metaheuristics (GA, SA, HC)** provide competitive solutions for medium-sized problems but incur significantly higher computational costs as problem size increases, with Genetic Algorithm being the most expensive.
- The **trade-off** between solution quality and **execution time** is pronounced across different scales.

Discussion & Conclusions

Randomized Greedy improves convergence

Time constraints make search difficult

Best approach = hybrid methods

References and Material Used

References & Literature

- Google Hash Code 2018 – Online Qualification Round (Problem Statement)
- Google Hash Code official competition materials
- Combinatorial Optimization literature
- Greedy Algorithms
- Hill Climbing
- Simulated Annealing
- Genetic Algorithm

Tools & Materials Used

- Course Materials
- Python
- Tkinter (GUI visualization)
- Visual Studio Code
- Standard Python libraries (math, random, sys)